[AUDITORIA DE SEGURANÇA DEFENSIVA • VERSÃO 1.0]

Relatório de Auditoria de Segurança — Loja SJ 3D

Revisão Exaustiva de Vulnerabilidades, Isolamento de Dados e RLS

Projeto:	Loja SJ 3D — Catálogo Oficial & Portal de Vendas
Organização / Loja:	SJ Novais e Cia Ltda (Umuarama - PR)
Data da Auditoria:	05 de Novembro de 2026
Classificação:	**CONFIDENCIAL / RESTRITO**
Escopo Auditado:	index.html, server.js, supabase/schema.sql, js/app.js, js/database.js, js/supabase-client.js, data/db.json, repositório git.
Stack Detectada:	Node.js nativo (HTTP Server), Supabase Auth / PostgreSQL 15+ (PostgREST), Vanilla JS / HTML5 / CSS3, Base JSON local (data/db.json).

Nota Metodológica & Mapeamento para a Stack

A auditoria cobriu cinco dimensões críticas de segurança da informação, mapeadas com exatidão para a arquitetura híbrida do projeto (Node.js nativo + Supabase Cloud + Vanilla JS):

1. Banco Sem Tranca (Isolamento de Tenant/Dono): Verificação de políticas Row Level Security (RLS) no Supabase (cláusulas USING/WITH CHECK e concessões à role authenticated) e ausência de autenticação/filtro por usuário nas rotas REST do server.js.

2. Permissão Definida no Navegador: Checagem de operações privilegiadas (cadastro de vendedor, edição de preços/catálogo, importação de base) onde o frontend oculta elementos visualmente via CSS/hash (#admin), mas os endpoints correspondentes não validam o papel de administrador no servidor.

3. IDOR: Rastreamento sistemático de todos os parâmetros :id de rotas no backend e mutações via PostgREST sem validação de posse.

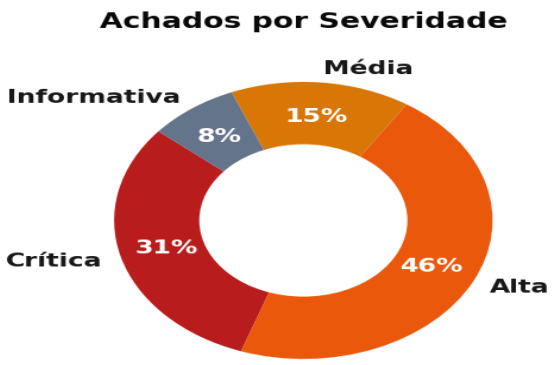
4. Chaves Expostas (Hardcode): Varredura de credenciais estáticas, senhas em texto puro, segredos no Git (PAT) e backdoors de bypass de autenticação.

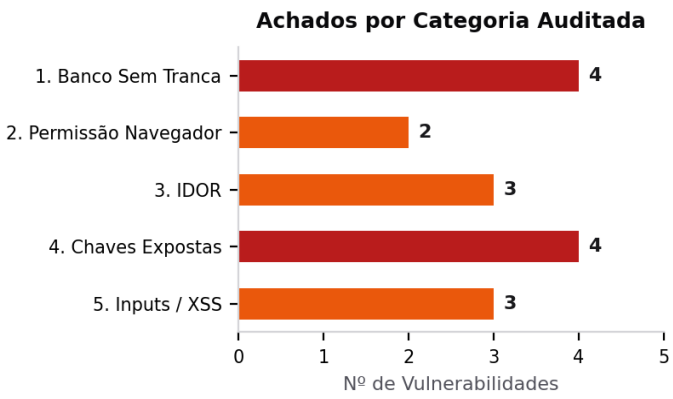
5. Inputs Sem Tratamento (XSS): Inspeção de inserções diretas em innerHTML sem sanitização prévia.

1. Resumo Executivo

Foram identificadas **13 vulnerabilidades** na aplicação, com prevalência de falhas críticas de **Isolamento de Dados no Supabase (RLS permissivo demais)**, **falta de autenticação nas rotas da API Node.js**, **backdoor de credenciais no frontend** e **DOM XSS por concatenação de inputs no dropdown de busca**. Apesar disso, foram identificados pontos fortes relevantes de arquitetura defensiva já presentes no código.

4	6	2	1	6
CRÍTICA	ALTA	MÉDIA	INFORMATIVA	PONTOS FORTES





2. Pontos Fortes e Pontos Fracos

Pontos Fortes Identificados (Proteções Ativas Verificadas):

• **Prevenção Contra Path Traversal no Servidor (server.js:444-448):** Validação estrita via filePath.startsWith(ROOT), bloqueando tentativas de navegação de diretório maliciosa (ex: ../../etc/passwd).
• **Proteção Contra DoS por Payload Gigante (server.js:106-110):** Limitação rígida de corpo de requisição em 10 MB (body.length > 1e7) com destruição imediata do socket.
• **Isolamento Estrito em Atualização de Perfil Supabase (schema.sql:239-245):** A política de atualização de perfis_usuarios valida explicitamente (select auth.uid()) = id, impedindo edição de perfis de terceiros.
• **Filtro de Catálogo Inativo para Usuários Anônimos (schema.sql:187-192):** Apenas produtos com ativo = true são visíveis publicamente na API.
• **Tratamento Completo de CORS Preflight (server.js:123-132):** Suporte a requisições HTTP OPTIONS com resposta 204 e cabeçalhos apropriados.
• **Gestão Segura de Sessão Supabase no Navegador (js/supabase-client.js:64-70):** Configuração nativa com persistSession: true e autoRefreshToken: true via SDK oficial.

Pontos Fracos Centrais (Riscos Sistêmicos):

• **Concessão RLS Irrestrita para 'authenticated' no Supabase:** Clientes cadastrados possuem o papel authenticated e, pelas regras atuais, herdam permissão para ler todos os clientes, alterar produtos e apagar orçamentos alheios.
• **Ausência Total de Autenticação nas Rotas REST Locais (server.js):** As rotas de dump (/api/db/export), restauração (/api/db/import) e mutação de catálogo operam sem checagem de token ou sessão.
• **Backdoor de Autenticação em Código-Fonte (js/database.js:273-279):** Verificação de credencial fixa (contato@gruposj.com.br / 123456) que promove qualquer usuário a Administrador no frontend.
• **DOM XSS em Busca ao Vivo (js/app.js:7057):** Interpolação direta de termoRaw em searchLiveDropdown.innerHTML sem escape de entidades.

3. Tabela de Achados Detalhados por Categoria

Abaixo estão todas as 13 vulnerabilidades verificadas diretamente no código real, com arquivo, linha(s), trecho explorável e justificativa técnica.

Sev.	Arquivo:Linha	Categoria & Descrição Técnica do Achado	Impacto & Condição
CRÍTICA	`supabase/schema.sql L273-298`	**RLS em public.orcamentos sem filtro de inquilino/dono.** Concede SELECT, INSERT, UPDATE, DELETE para authenticated com USING (true). Clientes cadastrados têm role authenticated e conseguem ler/modificar orçamentos de todos.	Vazamento massivo e adulteração de propostas comerciais por qualquer cliente logado.
ALTA	`supabase/schema.sql L246-271`	**RLS em public.clientes expõe dados pessoais a clientes.** Concede SELECT, UPDATE, DELETE para authenticated com USING (true).	Exposição de CPF/CNPJ, WhatsApp e endereços de toda a carteira de clientes da loja.
ALTA	`server.js L280-285, L359-364`	**Endpoints GET /api/orcamentos e /api/clientes sem autenticação.** Retornam todo o conteúdo de db.orcamentos e db.clientes para qualquer visitante.	Acesso irrestrito a dados comerciais sem exigir login ou chave de API.
CRÍTICA	`server.js L415-429`	**Exportação e Importação irrestrita do banco (/api/db/export e /api/db/import).** Permite download completo do db.json e substituição arbitrária da base inteira.	Comprometimento total da integridade e confidencialidade da base de dados.
CRÍTICA	`server.js L388-412`	**Edição e exclusão de catálogo (POST /api/catalogo e DELETE /api/catalogo) sem checagem de role.** O frontend esconde a UI em #admin, mas o backend aceita mutações abertas.	Qualquer visitante pode alterar preços, produtos ou apagar o catálogo da loja.
ALTA	`supabase/schema.sql L179-185, 199-217`	**Escrita de configurações e produtos no Supabase aberta a qualquer cliente.** Políticas usam to authenticated using (true) sem validar se perfis_usuarios.tipo = 'admin'.	Cliente comum pode executar inserts e updates em produtos e configuracoes_site.
ALTA	`server.js L331-356`	**IDOR em DELETE /api/orcamentos/:id e PATCH /status.** Parâmetro :id obtido da URL sem checar se o orçamento pertence ao chamador.	Exclusão e troca de status arbitrária de qualquer orçamento conhecendo seu ID.
ALTA	`server.js L286-327`	**IDOR em POST /api/orcamentos via body.** Sobrescreve propostas comerciais existentes passando o id de terceiros no JSON.	Adulteração de valores e itens de pedidos criados por outros clientes ou vendedores.
ALTA	`supabase/schema.sql L287-297`	**IDOR direto em public.orcamentos via SDK Supabase.** Atualização e exclusão de qualquer registro via PostgREST por cliente autenticado sem checagem de auth.uid() = cliente_id.	Exclusão e adulteração de orçamentos diretamente pelo cliente HTTP.

CRÍTICA	`.git/config` L7 (remoto origin)	**GitHub Personal Access Token (PAT) gravado na URL remota.** Token clássico com permissão de escrita no repositório exposto em arquivo de configuração local.	Sequestro do repositório remoto caso a pasta .git seja exposta ou compartilhada.
CRÍTICA	`js/database.js` L273-279	**Backdoor de login hardcoded no frontend.** Condição emailLimpo === 'contato@gruposj.com.br' && senhaLimpa === '123456' concede sessão de Admin imediatamente.	Bypass de autenticação universal diretamente no JavaScript do cliente.
ALTA	`server.js` L48-58 `data/db.json` L48-58	**Senha de administrador em texto puro (senhaHash: '123456').** Ausência de algoritmo de derivação segura de chave (bcrypt, Argon2 ou scrypt).	Comprometimento imediato da senha caso o arquivo db.json seja lido.
INFO	`js/supabase-client.js` L28-30	**Chave anônima pública do Supabase embutida no frontend.** A chave é pública por desenho, porém amplia os achados 1.1 e 1.2 caso o RLS permaneça furado.	Vetor de exploração direto contra as tabelas sem RLS restritivo.
ALTA	`js/app.js` L7057	**DOM XSS direto via busca rápida:** searchLiveDropdown.innerHTML = ... "${termoRaw}". Input do usuário injetado diretamente no DOM.	Execução de scripts arbitrários no contexto da sessão do cliente/vendedor.
MÉDIA	`js/app.js` L184	**Injeção HTML em notificações toast:** spanTxt.innerHTML = mensagem;. Parâmetros dinâmicos (nome de produtos e mensagens) injetados sem escape.	XSS induzido através de nomes maliciosos de produtos cadastrados.
MÉDIA	`js/app.js` L1007-1015, L1149-1175	**Ausência de sanitizador HTML (DOMPurify) na renderização de templates.** Dados de usuário (nome, telefone, foto) injetados via template strings diretamente no DOM.	Risco de Stored XSS persistente na visualização de perfil e pedidos.

4. Recomendações Priorizadas de Correção

As recomendações a seguir devem ser executadas em ondas estruturadas por prioridade de risco.

Prioridade	Ação Recomendada	Complexidade
P1 (Imediata)	**Blindar RLS em orcamentos e clientes no Supabase:** Criar função auth.is_admin() verificando perfis_usuarios.tipo = 'admin'. Clientes comuns só podem ler e criar orçamentos onde cliente_id = auth.uid(). **Revogar e rotacionar o GitHub PAT** embutido no .git/config. **Remover o backdoor** em js/database.js:273-279.	Média (1-2 dias)
P2 (Curto Prazo)	**Autenticar rotas de escrita e dump no server.js:** Adicionar middleware de token de autorização (Bearer token) ou remover /api/db/export e /api/db/import em ambiente produtivo. **Corrigir DOM XSS em js/app.js:7057:** Escapar termoRaw antes de injetar no innerHTML ou utilizar textContent. **Criptografar senhas locais** no db.json usando bcryptjs.	Baixa (1 dia)
P3 (Médio Prazo)	**Integrar DOMPurify globalmente:** Importar DOMPurify para higienizar qualquer template dinâmico renderizado via innerHTML. **Configurar Content Security Policy (CSP):** Definir cabeçalho Content-Security-Policy restritivo bloqueando scripts inline e fontes não autorizadas.	Média (2 dias)

5. Issues Acionáveis para o GitHub

Copie e cole o conteúdo de cada bloco delimitado diretamente como uma nova issue no repositório GitHub do projeto.

````
--- ISSUE 1 ---
# [Segurança] Blindar RLS em orçamentos e clientes no Supabase para isolar dados de clientes

**Labels:** `security`, `critical`, `supabase`, `database`

### Descrição do Problema
As tabelas `public.orcamentos` e `public.clientes` no Supabase possuem políticas de Row Level Security
(RLS) permissivas demais. Clientes que se cadastram no site recebem a role `authenticated`. Pelas políticas
atuais, qualquer usuário `authenticated` possui permissão irrestrita (`USING (true)`) para ler, criar,
editar e excluir registros de todos os clientes e orçamentos da loja.

### Evidência no Código
Arquivo: `supabase/schema.sql` (Linhas 246-271 e 273-298):
```sql
create policy "Apenas vendedores autenticados podem ler orçamentos"
on public.orcamentos for select to authenticated using (true);

create policy "Apenas vendedores autenticados podem ler clientes"
on public.clientes for select to authenticated using (true);
```

### Impacto
Vazamento em massa de dados protegidos pela LGPD (nomes, CPFs/CNPJs, WhatsApp, histórico de compras) e
possibilidade de exclusão/alteração arbitrária de pedidos da loja inteira por qualquer cliente cadastrado.

### Sugestão de Correção
1. Criar uma função auxiliar no Supabase:
```sql
create or replace function public.is_admin_ou_vendedor()
returns boolean language sql security definer as $$
select exists (
select 1 from public.perfis_usuarios
where id = auth.uid() and tipo in ('operador', 'admin') and ativo = true
);
$$;
```
2. Restringir orçamentos: vendedores lêem tudo; clientes lêem apenas onde `cliente_id = auth.uid()::text`.

### Critérios de Aceite
- [ ] Cliente autenticado comum só consegue listar seus próprios orçamentos via
`supabase.from('orcamentos').select()`.
- [ ] Tentativa de ler orçamentos de outros clientes retorna zero linhas.
- [ ] Vendedores autenticados continuam acessando todos os orçamentos no painel `/admin`.
--- FIM ISSUE 1 ---
````

```
--- ISSUE 2 ---
# [Segurança] Autenticar rotas administrativas e desativar export/import público no server.js

**Labels:** `security`, `critical`, `backend`, `node`

### Descrição do Problema
O backend local em `server.js` expõe endpoints administrativos sem qualquer verificação de autenticação ou
token:
- `GET /api/db/export`: Faz o dump de todo o banco JSON (incluindo usuários e senhas).
- `POST /api/db/import`: Substitui a base de dados inteira.
- `POST /api/catalogo` e `DELETE /api/catalogo`: Modificam ou resetam produtos e preços do catálogo.
- `GET /api/clientes` e `GET /api/orcamentos`: Listam todos os dados cadastrais.

### Evidência no Código
Arquivo: `server.js` (Linhas 388-429):
```javascript
if (pathname === "/api/catalogo") {
if (req.method === "POST") { ... salvarBanco(db); }
}
if (pathname === "/api/db/export") { responderJson(res, 200, db); }
```

Impacto
Qualquer visitante na mesma rede ou com acesso à URL da API pode apagar o catálogo, baixar todos os dados
pessoais e substituir a base da loja.

Sugestão de Correção
1. Criar um middleware de validação de cabeçalho `Authorization: Bearer <ADMIN_SECRET_KEY>`.
2. Em ambiente produtivo, desabilitar totalmente os endpoints `/api/db/export` e `/api/db/import`.

Critérios de Aceite
- [ ] Requisições para `/api/catalogo` sem token retornam HTTP 401 Unauthorized.
- [ ] Endpoint `/api/db/export` bloqueia chamadas não autorizadas.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Segurança] Remover backdoor de credenciais hardcoded e aplicar hash em senhas

**Labels:** `security`, `critical`, `auth`

### Descrição do Problema
O arquivo `js/database.js` possui um bypass de autenticação hardcoded no frontend que autentica como
Administrador qualquer tentativa de login com o e-mail `contato@gruposj.com.br` e senha `123456`, mesmo se
o usuário tiver alterado sua senha. Além disso, as senhas no `server.js` e `data/db.json` estão salvas em
texto puro (`senhaHash: "123456"`).

### Evidência no Código
Arquivo: `js/database.js` (Linhas 273-279):
```javascript
if (emailLimpo === "contato@gruposj.com.br" && senhaLimpa === "123456") {
state.usuarioAtual = { ...USUARIO_PADRAO, ultimoLogin: new Date().toISOString() };
return { sucesso: true, usuario: state.usuarioAtual };
}
```

Impacto
Acesso não autorizado ao painel administrativo por qualquer pessoa inspecionando o código do frontend.

Sugestão de Correção
1. Remover completamente o bloco de bypass condicional em `js/database.js`.
2. No `server.js`, utilizar biblioteca `bcryptjs` para comparar senhas com hash salt.

Critérios de Aceite
- [ ] O código-fonte não possui credenciais ou e-mails embutidos em condições `if`.
- [ ] Tentativa de login com senha não cadastrada é rejeitada.
- [ ] `senhaHash` no `db.json` contém hash bcrypt e não texto puro.
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
# [Segurança] Rotacionar GitHub Personal Access Token (PAT) exposto no .git/config

**Labels:** `security`, `high`, `git`, `devops`

### Descrição do Problema
A URL remota do repositório Git contém um token de acesso pessoal (PAT) clássico do GitHub em texto puro no arquivo `.git/config`. Se este arquivo ou o ambiente for compartilhado, o token confere acesso direto de leitura e escrita ao repositório no GitHub.

### Evidência no Código
Arquivo: `.git/config`:
```text
url = https://ghp_REDACTED_SECRET_TOKEN@github.com/addesigner2026/lojasj3d-site.git
```

Impacto
Possibilidade de sequestro do repositório, commit de código malicioso e vazamento de histórico.

Sugestão de Correção
1. No painel do GitHub, revogar imediatamente o token exposto.
2. Atualizar a URL remota local para usar SSH ou Git Credential Manager:
```bash
git remote set-url origin https://github.com/addesigner2026/lojasj3d-site.git
```

Critérios de Aceite
- [ ] O token anterior foi revogado no painel do GitHub.
- [ ] `.git/config` não contém tokens na URL remota.
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
# [Segurança] Corrigir DOM XSS em dropdown de busca e sanitizar interpolações HTML

**Labels:** `security`, `high`, `xss`, `frontend`

### Descrição do Problema
No arquivo `js/app.js`, a função `renderizarSugestoesBuscaAoVivo` concatena diretamente o parâmetro `termoRaw` dentro de `searchLiveDropdown.innerHTML` quando nenhum resultado é encontrado. Além disso, as mensagens de notificação toast e templates de perfil utilizam `innerHTML` sem biblioteca de higienização.

### Evidência no Código
Arquivo: `js/app.js` (Linhas 7055-7058):
```javascript
searchLiveDropdown.innerHTML = `
<div class="search-live-empty">
<span>Nenhum item encontrado para <strong>"${termoRaw}"</strong>.</span>
```

Impacto
Execução de scripts maliciosos (XSS) no navegador do usuário, possibilitando roubo de tokens de sessão do Supabase no `localStorage`.

Sugestão de Correção
1. Criar uma função utilitária de escape de HTML:
```javascript
function escaparHtml(str) {
return String(str || '').replace(/[&<>"']/g, (m) => ({
'&': '&', '<': '<', '>': '>', '"': '"', "'": '''
})[m]);
}
```
2. Aplicar `escaparHtml(termoRaw)` em todas as interpolações de template string.
3. Importar a biblioteca `DOMPurify` para sanitizar conteúdos dinâmicos complexos.

Critérios de Aceite
- [ ] Digitar `<img src=x onerror=alert(1)>` no campo de busca renderiza o texto puro sem executar JavaScript.
- [ ] Toasts e nomes de perfil não interpretam tags HTML maliciosas.
--- FIM ISSUE 5 ---
```